

CS 5250 Final Project

Mihail Chitorog

BB84 Quantum Key Distribution on IBM Quantum Hardware

GitHub Link:

<https://github.com/mikeleon001/cs5250-quantum-final>

Youtube Link:

<https://www.youtube.com/watch?v=i4ofZGBaRTc>

quantum_bb84_ibm.py

```
# BB84 Quantum Key Distribution – Runs on Real IBM Quantum Hardware
# CS 5250 Advanced Computer Architecture – Final Project
# Mihail Chitorog
```

```
from qiskit import QuantumCircuit, transpile
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
import random

# -----
# CONNECT TO IBM QUANTUM
# -----
print("Connecting to IBM Quantum...")
service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)
print(f"Connected! Running on: {backend.name} ({backend.num_qubits} qubits)")

# -----
# HELPER: Run circuits on REAL IBM hardware
# -----
def run_on_ibm(circuits, shots=1):
    """
    Submitting quantum circuits to real IBM hardware and get results.
    Transpiling converts our abstract circuit to the physical gate set
    the specific quantum computer supports.
    """
    transpiled = transpile(circuits, backend=backend)
    sampler = Sampler(backend)
    job = sampler.run(transpiled, shots=shots)
```

```

print(f"  Job submitted: {job.job_id()}")
print(f"  Waiting for IBM quantum computer to process...")
result = job.result()
return result

# -----
# ALICE: Prepare qubits
# -----
def alice_prepare_qubit(bit, basis):
    """
    Alice encodes one classical bit into a qubit.

    Two bases (like two different 'languages'):
    + basis (rectilinear): 0 = |0>, 1 = |1>
    x basis (diagonal):   0 = |+>, 1 = |->

    Same basis → correct result.
    Different basis → random garbage (50/50).
    This is the heart of BB84.
    """
    qc = QuantumCircuit(1, 1)
    if bit == 1:
        qc.x(0)
    if basis == 'x':
        qc.h(0)
    return qc

# -----
# BOB: Measure qubits
# -----
def bob_measure(qc, basis):
    """
    Bob measures in his randomly chosen basis.
    Match → correct bit. Mismatch → random bit.
    """
    qc = qc.copy()
    if basis == 'x':
        qc.h(0)
    qc.measure(0, 0)
    return qc

# -----
# EVE: Intercept and resend
# -----
def eve_intercept(qc):
    """
    Eve guesses a random basis to measure in.
    Wrong guess → qubit is disturbed irreversibly.
    Alice and Bob detect this as errors.
    This is the quantum no-cloning theorem in action.
    """
    eve_basis = random.choice(['+', 'x'])
    qc_eve = qc.copy()
    if eve_basis == 'x':
        qc_eve.h(0)
    qc_eve.measure(0, 0)

    # Eve resends in her measured basis (damage already done)

```

```

    eve_resend = QuantumCircuit(1, 1)
    if eve_basis == 'x':
        eve_resend.h(0)
    return eve_resend

# -----
# MAIN BB84 SIMULATION ON IBM HARDWARE
# -----
def run_bb84_ibm(num_bits=8, eve_present=False):
    """
    Running BB84 on real IBM quantum hardware.
    We batch all circuits together to minimize IBM job submissions.
    """
    print("\n" + "=" * 55)
    print("        BB84 Quantum Key Distribution Protocol")
    print("        Running on REAL IBM Quantum Hardware")
    print("=" * 55)
    print(f"Qubits: {num_bits} | Eve present: {eve_present}")
    print(f"Backend: {backend.name}")
    print("=" * 55)

    # Alice generates random bits and bases
    alice_bits = [random.randint(0, 1) for _ in range(num_bits)]
    alice_bases = [random.choice(['+', 'x']) for _ in range(num_bits)]
    bob_bases = [random.choice(['+', 'x']) for _ in range(num_bits)]

    # Build all circuits
    print("\n[1] Building quantum circuits...")
    circuits = []
    for i in range(num_bits):
        qc = alice_prepare_qubit(alice_bits[i], alice_bases[i])
        if eve_present:
            qc = eve_intercept(qc)
        qc = bob_measure(qc, bob_bases[i])
        circuits.append(qc)

    # Print one example circuit
    print("\n    Example circuit (qubit 0):")
    print(circuits[0].draw(output='text'))

    # Submit ALL circuits to IBM in one batch job
    print(f"\n[2] Submitting {num_bits} circuits to {backend.name}...")
    result = run_on_ibm(circuits, shots=1)

    # Extract results
    bob_results = []
    for i in range(num_bits):
        counts = result[i].data.c.get_counts()
        bit = int(list(counts.keys())[0])
        bob_results.append(bit)

    # -----
    # Sifting — keep matching bases
    # -----
    print("\n[3] Sifting — comparing bases publicly...")
    sifted_alice = []
    sifted_bob = []

```

```

for i in range(num_bits):
    if alice_bases[i] == bob_bases[i]:
        sifted_alice.append(alice_bits[i])
        sifted_bob.append(bob_results[i])

print(f"    Matching bases: {len(sifted_alice)}/{num_bits} bits kept")

# -----
# Error checking - detect Eve
# -----
print("\n[4] Checking for eavesdropping...")
check_size = min(6, len(sifted_alice))
errors = sum(1 for i in range(check_size)
             if sifted_alice[i] != sifted_bob[i])
error_rate = errors / check_size if check_size > 0 else 0
print(f"    Error rate: {errors}/{check_size} = {error_rate:.0%}")

# Final key
final_key_alice = sifted_alice[check_size:]
final_key_bob   = sifted_bob[check_size:]

print("\n[5] Final Results:")
print(f"    Alice's key : {final_key_alice}")
print(f"    Bob's key   : {final_key_bob}")

if error_rate > 0.15:
    print("\n    HIGH ERROR RATE - Eve likely detected! Discarding key.")
elif final_key_alice == final_key_bob:
    print("\n    Keys match! Secure channel established.")
    print(f"    Shared key: {final_key_alice}")
else:
    print("\n    Keys don't match - possible interference detected.")

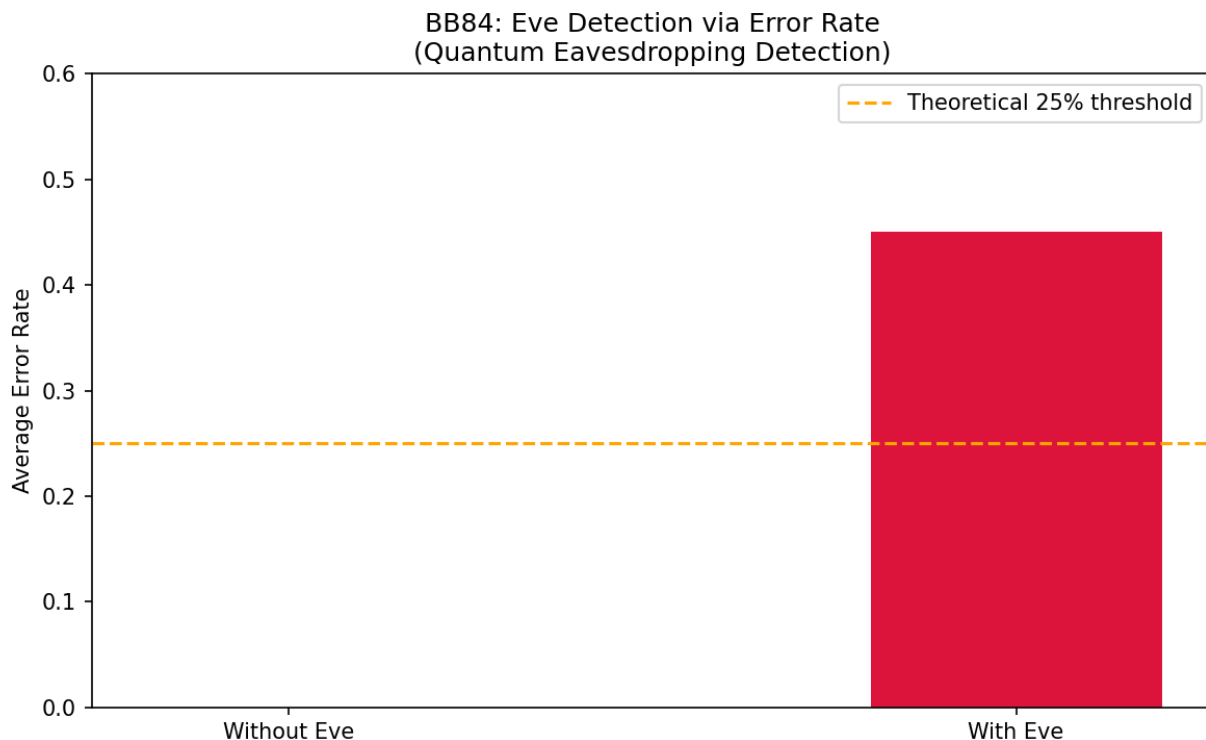
return final_key_alice, error_rate

# -----
# RUN BOTH SCENARIOS ON REAL HARDWARE
# -----
if __name__ == "__main__":
    print("\n-----")
    print("    SCENARIO 1: Secure transmission (no eavesdropper)")
    print("-----")
    key1, err1 = run_bb84_ibm(num_bits=20, eve_present=False)

    print("\n-----")
    print("    SCENARIO 2: Eve is intercepting the channel")
    print("-----")
    key2, err2 = run_bb84_ibm(num_bits=20, eve_present=True)

    print("\n-----")
    print("    SUMMARY")
    print("-----")
    print(f"    Scenario 1 error rate: {err1:.0%} (expected ~0%)")
    print(f"    Scenario 2 error rate: {err2:.0%} (expected ~25%)")
    print(f"\n    Both scenarios ran on real IBM quantum hardware: {__backend__.__name__}")
    print("    This is NOT a simulation - these are real quantum measurements!")

```



quantum_rng.py

Quantum Random Number Generator using IBM Qiskit
CS 5250 Final Project – Mihail Chitorog

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
```

```
def quantum_random_number(num_bits=8):
    """
```

Generate a truly random number using quantum superposition.

A classic computers use pseudo-random algorithms (deterministic).
A qubit in superposition has NO predetermined outcome. It is
genuinely 50/50 until measured. This is TRUE randomness.
"""

```
# Create a circuit with num_bits qubits and num_bits classical bits
qc = QuantumCircuit(num_bits, num_bits)
```

```

# Apply Hadamard gate to every qubit
# H gate puts qubit into superposition:  $|0\rangle \rightarrow (|0\rangle + |1\rangle)/\sqrt{2}$ 
# Before measurement, the qubit is BOTH 0 and 1 simultaneously
for i in range(num_bits):
    qc.h(i)

# Measure all qubits → collapses superposition to 0 or 1
qc.measure(range(num_bits), range(num_bits))

return qc

def run_and_display(num_bits=8, shots=1024):
    """
    Running the circuit on a local quantum simulator and display results.
    shots = how many times we repeat the experiment
    """

    qc = quantum_random_number(num_bits)

    # Draw the circuit so we can see it
    print("=== Quantum Circuit ===")
    print(qc.draw(output='text'))

    # Use local Aer simulator
    simulator = AerSimulator()

    # Run the circuit
    job = simulator.run(qc, shots=shots)
    result = job.result()
    counts = result.get_counts()

    # Pick one random result and convert binary → integer
    # In a real run, each shot gives a different random bitstring
    sample_bitstring = list(counts.keys())[0]
    random_number = int(sample_bitstring, 2)

    print(f"\n=== Results ({shots} shots) ===")
    print(f"Sample random number (0-{2**num_bits - 1}): {random_number}")
    print(f"All measurement outcomes: {len(counts)} unique values")
    print(f"\nTop 5 outcomes:")
    sorted_counts = sorted(counts.items(), key=lambda x: x[1], reverse=True)
    for bitstring, count in sorted_counts[:5]:
        print(f"  |{bitstring}| → {int(bitstring,2):3d}  ({count} times)")

    # Plot histogram of results
    fig = plot_histogram(counts, title="Quantum Random Number Distribution")
    plt.tight_layout()
    plt.savefig("quantum_rng_histogram.png", dpi=150)
    plt.show()
    print("\nHistogram saved as quantum_rng_histogram.png")

    return random_number, counts

def compare_classical_vs_quantum():
    """
    Showing the conceptual difference for the presentation.
    Classical: deterministic algorithm pretending to be random.
    Quantum: genuinely non-deterministic at the physics level.
    """

```

```

"""
import random

print("=== Classical RNG (pseudo-random) ===")
print("Uses a seed + math formula. Given same seed → SAME sequence
always.")
random.seed(42)
classical = [random.randint(0, 255) for _ in range(5)]
print(f"Seed 42 → {classical}")
random.seed(42)
classical_repeat = [random.randint(0, 255) for _ in range(5)]
print(f"Seed 42 again → {classical_repeat} ← identical!")

print("\n=== Quantum RNG (truly random) ===")
print("No seed. No formula. Physics itself decides.")
print("Running quantum circuit...")
num, _ = run_and_display(num_bits=8, shots=5)
print("Each run produces a genuinely different outcome.")

if __name__ == "__main__":
    compare_classical_vs_quantum()

```

